Lập trình mạng

Socket

TS.GVC. Nguyễn Văn Khanh
PLC & IIoT Lab

Cần Thơ, ngày 28 tháng 7 năm 2025

Thư viện Socket.IO trên ESP32

- Tham khảo phần giới thiệu trong thư viện khi cài đặt

ESP32 socket.IO client đơn giản

- Để phù hợp với client, máy chủ viết bằng JS cần cài đặt socket.IO phiên bản 2.4.1 bằng lệnh sau

```
npm install socket.io@2.4.1
```

- Tạo và thực thi máy chủ lắng nghe tại cổng 3000:

```
const io = require("socket.io")(3000, {
    origins: "*:*"
});

io.on("connection", (socket) => {
    console.log("ESP32 connected:", socket.id);

    socket.on("hello", (msg) => {
        console.log("ESP32 says:", msg);
    });
});
```

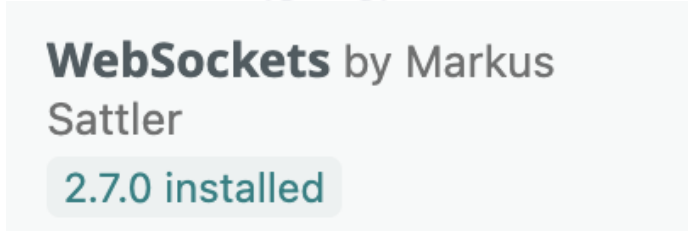
package.json

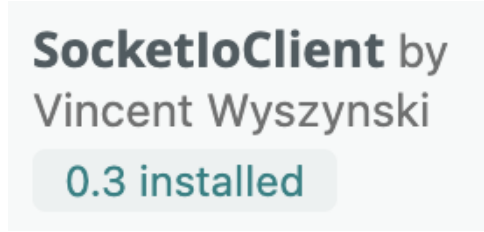
```
{
  "dependencies": {
    "socket.io": "^2.4.1"
  }
}
```

ESP32 client

- Khởi động socket

```
// Địa chỉ server Socket.IO
const char* host = "192.168.1.36"; // IP máy chủ, kiểm tra
const uint16_t port = 3000;
SocketIoClient socket;
void setup() {
    //Khởi động WiFi và các mô-đun liên quan
    socket.begin(host, port);
    socket.on("connect", [] (const char * payload, size_t length){
        Serial.println("Connected to Socket.IO server!");
        socket.emit("hello", "\"Hello from ESP32\"");
    });
}
void loop() {
    socket.loop();
    delay(100);
}
```





- Cài thêm thư viện: tìm và xoá hàm hexdump trong tập tin SocketIoClient.cpp

Client gửi dữ liệu về JS Server

- Bổ sung thêm mã lệnh cho máy chủ JS

```
const io = require("socket.io")(3000, {
    origins: "*:*"
});

io.on("connection", (socket) => {
    console.log("ESP32 connected:", socket.id);

    socket.on("hello", (msg) => {
        console.log("ESP32 says:", msg);
    });

    socket.on("esp-data", (dat) => {
        console.log("esp-data:", dat);
    });
});
```

Thêm mã lệnh cho ESP32 client

- Mã lệnh ESP32 client được cập nhật

```
void loop() {
    socket.loop();

    // Gửi dữ liệu mỗi 5 giây
    static unsigned long lastSend = 0;
    if (millis() - lastSend > 3000) {
        lastSend = millis();
        String dat = "{\"value\":25.5}";
        socket.emit("esp-data", dat.c_str());
        Serial.println("Sent data");
    }
}
```

Gửi dữ liệu từ JS Server đến ESP client

- Chương trình JS Server bổ sung chức năng gửi biến count đến client mỗi 15s

```
const io = require("socket.io")(3000, {
    origins: "*:*"
});

io.on("connection", (socket) => {
    console.log("ESP32 connected:", socket.id);
    // chờ thông điệp chào từ ESP32
    socket.on("hello", (msg) => {
        console.log("ESP32 says:", msg);
    });
    // chờ dữ liệu từ ESP32
    socket.on(”esp-data", (dat) => {
        console.log(”esp-data:", dat);
    });
});

// Broadcast a control command to all clients every 15s (toggle)
let count = 0;
setInterval(() => {
    io.emit(”s-data", JSON.stringify({"s-data": count++ }));
    console.log("Broadcast data =", count);
}, 15000);
```

Mã lệnh ESP32 client

- Chương trình client cập nhật:

```
// Địa chỉ server Socket.IO
const char* host = "192.168.1.36"; // IP máy chủ
const uint16_t port = 3000;
SocketIoClient socket;
void setup() {
    // Khởi động các mô-đun cần thiết
    socket.begin(host, port);
    socket.on("connect", [] (const char * payload, size_t length) {
        Serial.println("Connected to Socket.IO server!");
        socket.emit("hello", "\"Hello from ESP32\"");
    });

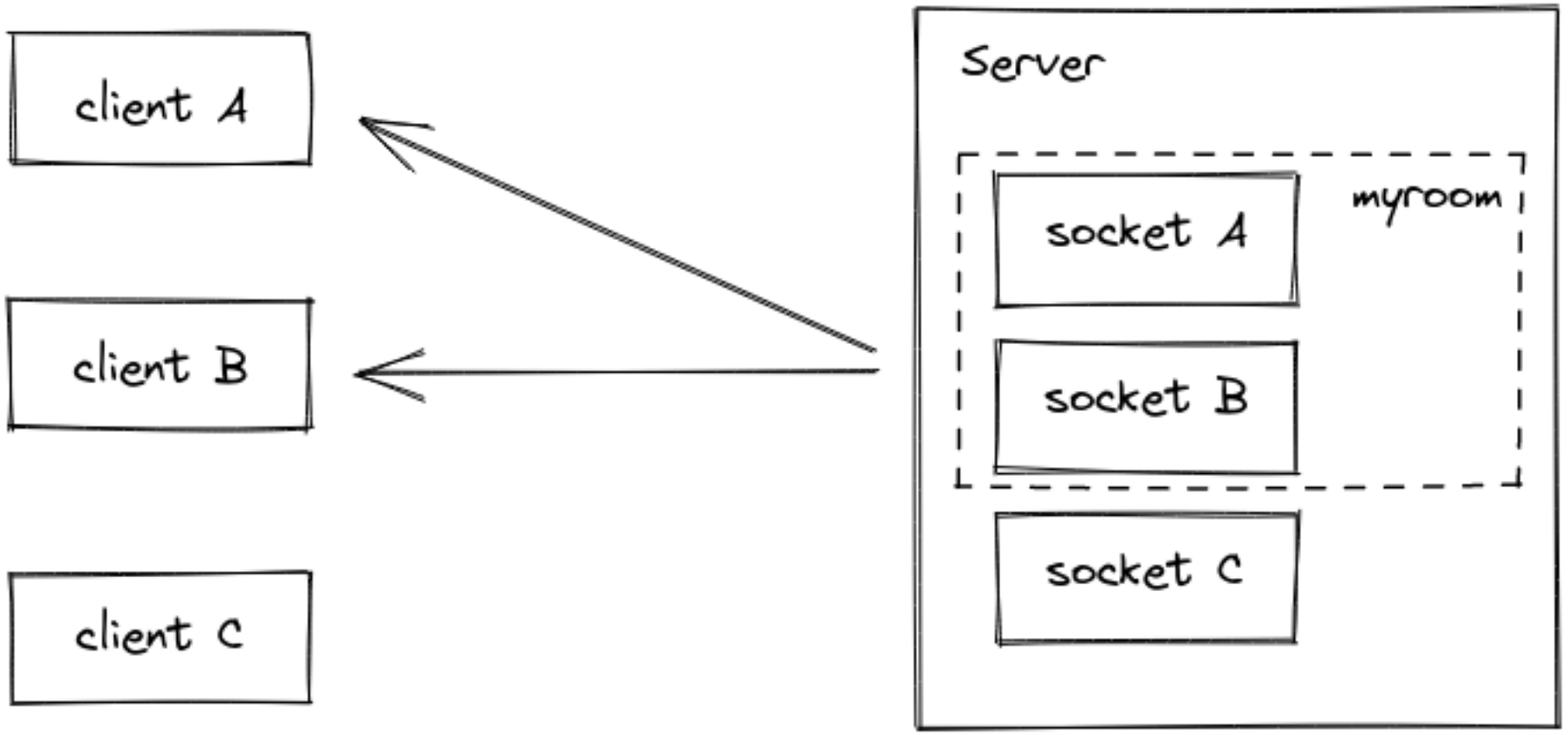
    socket.on("s-data", [] (const char * payload, size_t length) {
        Serial.print("Received data: ");
        Serial.println(payload);
        // Chèn đoạn xử lý JSON
    });
}
```

Bài tập

1. Đọc giá trị ADC gửi về máy chủ JS
2. Định kỳ bật tắt trạng thái của LED bằng lệnh từ máy chủ JS
3. Viết ứng dụng web đơn giản hiển thị dữ liệu từ ESP client
4. Thêm nút điều khiển trạng thái của LED trên ESP32

ROOM trong SocketIO

- Là một kênh cho phép các socket có thể tham gia (join) và rời khỏi (leave) một nhóm trao đổi thông tin quảng bá



https://socket.io/docs/v3/rooms/

Join và leave

- Join

```
io.on('connection', socket => {
  socket.join('some room');
});
```

Gửi thông điệp tới các socket trong cùng ROM

```
io.to('some room').emit('some event');
```

”to” và “in” có chức năng giống nhau

Gửi thông điệp tới các socket trong nhiều ROM

```
io.to('room1').to('room2').to('room3').emit('some event');
```

Chỉ định socket nhận thông điệp

```
io.on('connection', function(socket){
  socket.to('some room').emit('some event');
});
```

- Leave: cùng cú pháp với join và có thể gọi callback

Join và leave

- Join

```
io.on('connection', socket => {
  socket.join('some room');
});
```

Gửi thông điệp tới các socket trong cùng ROM

```
io.to('some room').emit('some event');
```

”to” và “in” có chức năng giống nhau

Gửi thông điệp tới các socket trong nhiều ROM

```
io.to('room1').to('room2').to('room3').emit('some event');
```

Chỉ định socket nhận thông điệp

```
io.on('connection', function(socket){
  socket.to('some room').emit('some event');
});
```

- Leave: cùng cú pháp với join và có thể gọi callback

Room mặc định

- Trong Socket.IO mỗi socket được định danh bởi một số mặc định, không thể đoán trước và duy nhất Socket#id
- Các socket sẽ tự động tham gia vào Room trùng tên với *id* của nó
- Dễ dàng truyền thông điệp tới các socket

```
io.on('connection', socket => {
  socket.on('say to someone', (id, msg) => {
    socket.to(id).emit('my message', msg);
  });
});
```

Một số trường hợp ứng dụng

- Quảng bá dữ liệu đến từng thiết bị

```
io.on('connection', async (socket) => {
  const userId = await computeUserIdFromHeaders(socket);

  socket.join(userId);

  // and then later
  io.to(userId).emit('hi');
});
```

- Thông báo về một thực thể

```
io.on('connection', async (socket) => {
  const projects = await fetchProjects(socket);

  projects.forEach(project => socket.join('project:' + project.id));

  socket.on('update project', async (payload) => {
    const project = await updateProject(payload);
    io.to('project:' + project.id).emit('project updated', project);
  });
});
```

Ngắt kết nối

- Lấy socket của Room đã tham gia để gọi sự kiện disconnect

```
io.on('connection', socket => {
  socket.on('disconnecting', () => {
    const rooms = Object.keys(socket.rooms);
    // the rooms array contains at least the socket ID
  });

  socket.on('disconnect', () => {
    // socket.rooms === {}
  });
});
```

‘Map’ trong JS

- Map là một cấu trúc dữ liệu dạng key-value:
  - Key có thể chưa một kiểu dữ liệu bất kỳ
  - Hỗ trợ phương thức thêm, xoá và duyệt phần tử
- Tạo Map:

```
let myMap = new Map();
```

```
let myMap = new Map([
  ['name', 'Alice'],
  ['age', 25],
  [true, 'boolean key'],
]);
```

- Thêm, hiệu chỉnh phần tử

```
myMap.set('name', 'Bob');   // Thêm mới hoặc ghi đè key 'name'
myMap.set(100, 'number key');
myMap.set({ id: 1 }, 'object key');
```

‘Map’ trong JS

- Đọc dữ liệu

```
console.log(myMap.get('name'));  // "Bob"
console.log(myMap.get(100));     // "number key"

// Nếu key không tồn tại → trả về undefined
console.log(myMap.get('notExist')); // undefined
```

- Kiểm tra key

```
console.log(myMap.has('name')); // true
console.log(myMap.has('age'));  // false (nếu đã xóa)
```

- Xoá dữ liệu

```
myMap.delete('name'); // xóa 1 phần tử
myMap.clear();        // xóa toàn bộ map
```

Duyệt phần tử trong 'Map'

- Duyệt bằng vòng lập for

```
for (let [key, value] of myMap) {
  console.log(key, value);
}
```

- Duyệt bằng forEach

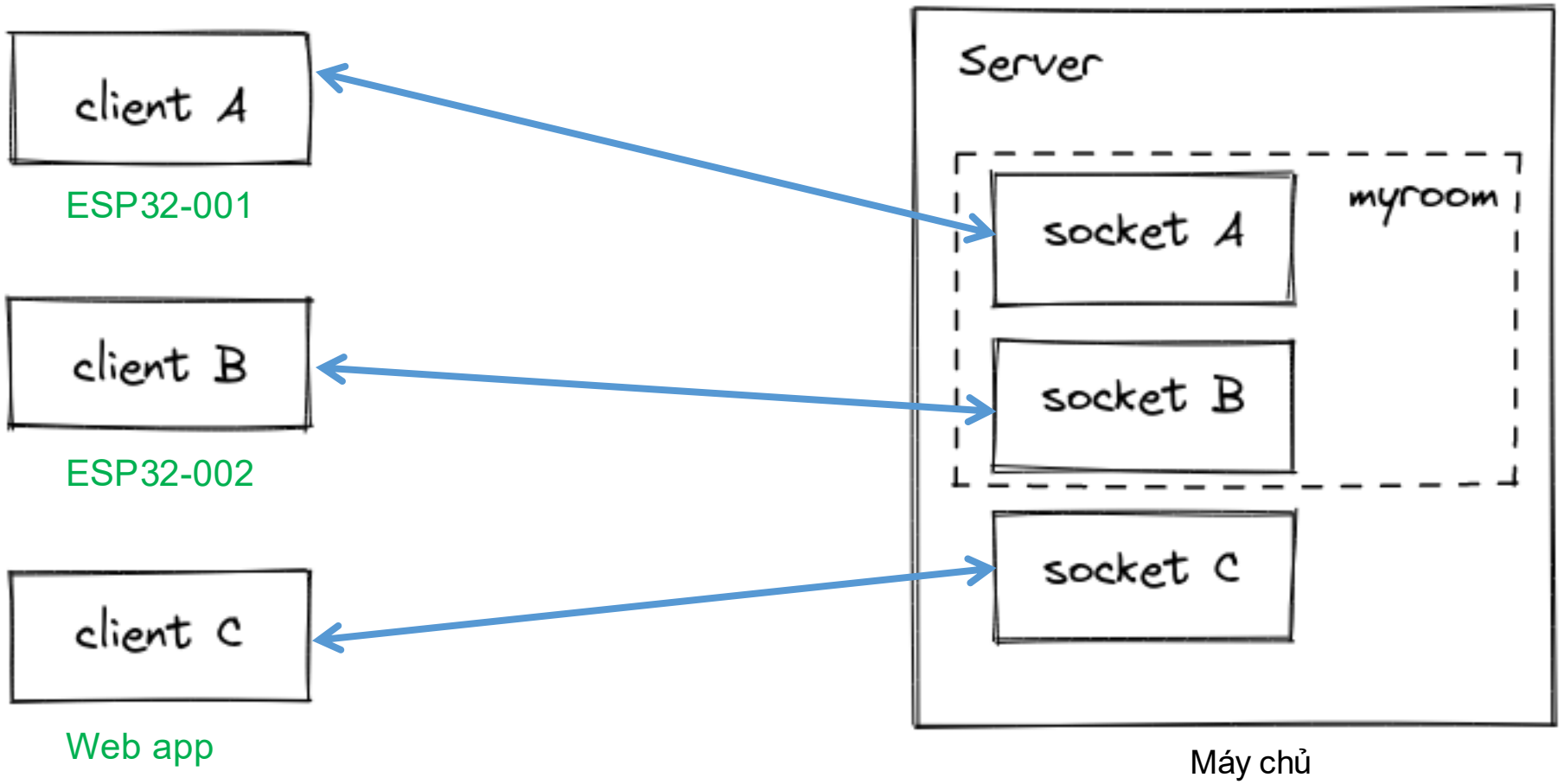
```
myMap.forEach((value, key) => {
  console.log(key, value);
});
```

- Duyệt bằng toán tử …

```
console.log([...myMap.keys()]);
console.log([...myMap.values()]);
console.log([...myMap.entries()]);
```

Ứng dung ROOM trong quản lý nhiều ESP32 client

- Các ESP32 client sẽ kết nối với máy chủ JS và tham gia vào một ROOM chung



Máy chủ JS

- Máy chủ JS có thể broadcast thông điệp đến tất cả các client hoặc từng client thông qua ROOM của nó

```
// Quản lý các thiết bị
const devices = new Map(); // deviceId -> { socketId, lastSeen, data }

io.on('connection', (socket) => {
  console.log('Client connected:', socket.id);
  // ESP32 registration with deviceId
  socket.on('register_device', (deviceId) => {
    if (!deviceId) {
      console.log('Device registration failed - no deviceId provided');
      return;
    }
    // Tham gia vào ROOM
    socket.join(`device_${deviceId}`);
    // Lưu thông tin thiết bị
    devices.set(deviceId, {
      socketId: socket.id,
      lastSeen: Date.now(),
      data: null
    });
    console.log(`ESP32 registered: ${deviceId} (${socket.id})`);
  });
```

Nhận dữ liệu từ ESP32

- Xử lý trạng thái định kỳ gửi về máy chủ JS

```
//xem cấu trúc payload trong chương trình ESP32
socket.on(
  'status', (payload) = > {
    let deviceId = null;
    //đọc device ID
    for (const[id, info] of devices) {
      if (info.socketId == = socket.id) {
        deviceId = id;
        break;
      }
    }
    if (!deviceId) {
      console.log('Trạng thái chưa được đăng ký:', socket.id);
      return;
    }
    devices.get(deviceId).data = payload;
    devices.get(deviceId).lastSeen = Date.now();
    console.log(`${deviceId}:`, devices?.get(deviceId).data);
    console.log(`Heartbeat of ${ deviceId:`, payload);
  });
```

Cập nhật trạng thái kết nối

- Xử lý trạng thái định kỳ gửi về từ ESP32 client

```
// Cleanup disconnected devices periodically
setInterval(() => {
  const now = Date.now();
  const timeout = 30000; // 30 seconds

  devices.forEach((info, deviceId) => {
    if (now - info.lastSeen > timeout) {
      console.log(`Device ${deviceId} timed out, removing...`);
      devices.delete(deviceId);
    }
  });
}, 15000); // Check every 15 seconds
```

Gửi lệnh đình kỳ đến ESP32

- Gửi lệnh điều khiển đến một thiết bị cụ thể dựa vào tên ROOM của nó

```
setInterval(() => {
  io.to(`device_ESP32_001`).emit('control', JSON.stringify({ cmd: 'toggle' }));
}, 3000);
```

Ví dụ

- JS Server side: thực hiện chức năng đăng ký thiết bị, chuyển thiết bị đến ROOM của nó

```
const express = require('express');
const http = require('http');
const socketIo = require('socket.io');
const server = http.createServer(app);


const app = express();
const io = socketIo(server, {
    origins: "*:*"
});


const PORT = 3000;
// Device management
// // deviceId -> { socketId, lastSeen, data}
const devices = new Map();
```

```
io.on('connection', (socket) => {
    console.log('Client connected:', socket.id);

    // ESP32 registration with deviceId
    socket.on('register_device', (deviceId) => {
        if (!deviceId) {
            console.log('Device registration failed - no deviceId
provided');
            return;
        }

        // Join device-specific room
        socket.join(`device_${deviceId}`);

        // Store device info
        devices.set(deviceId, {
            socketId: socket.id,
            lastSeen: Date.now(),
            data: null
        });

        console.log(`ESP32 registered: ${deviceId} (${socket.id})`);
        devices.forEach((value,key)=>{
            console.log(key, value);
        })
    });
});


server.listen(PORT, () => {
  console.log(`Multi-ESP32 Server running on http://localhost:${PORT}`);
  console.log('Waiting for ESP32 devices clients...');
});
```

Ví dụ

- ESP client side: kết nối đến server, gửi thông tin đăng ký thiết bị

```
#include <Arduino.h>
#include <ArduinoJson.h>
#include <WiFi.h>
#include <SocketIoClient.h>


// ================================
// DEVICE CONFIGURATION
// ================================
// Thay đổi DEVICE_ID cho từng ESP32 khác nhau
// ESP32_001, ESP32_002, ESP32_003, etc.
const String DEVICE_ID = "ESP32_002";


// WiFi Configuration
const char* ssid = “ssid";
const char* password = “pass";
bool isConnected = false;
// Server Configuration
// Thay bằng IP của máy chạy server
const char* host = "192.168.1.125";
const uint16_t port = 3000;
```

```
// Hardware Configuration
#define ledPin  2
// ================================
// GLOBAL VARIABLES
// ================================
SocketIoClient socket;
bool ledState = false;
unsigned long lastTelemetry = 0;
unsigned long lastStatus = 0;
bool isConnectedToServer = false;


void setup() {
  Serial.begin(115200);
  Serial.println("\n================================");
  Serial.println("Multi-ESP32 client System");
  Serial.println("Device ID: " + DEVICE_ID);
  Serial.println("================================");

  // Hardware initialization
  pinMode(0, INPUT);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  // Connect to WiFi
  connectWiFi();

  // Connect to Socket.IO server
  connectToServer();
  isConnected = true;
}
```

Ví dụ

- ESP client side: thực hiện chức năng đăng ký thiết bị, chuyển thiết bị đến ROOM của nó

```
void loop() {
  socket.loop();
  delay(100);
  // Check WiFi connection
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("📶 WiFi disconnected,
reconnecting...");
    connectWiFi();
  }

  // Send status every 30 seconds
  if (millis() - lastStatus > 5000) {
    // sendStatus();
    lastStatus = millis();
  }
}
```

```
void connectWiFi() {
  Serial.print("Connecting to WiFi: " + String(ssid));
  WiFi.begin(ssid, password);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWiFi Connected!");
    Serial.println("IP Address: " + WiFi.localIP().toString());
    Serial.println("MAC Address: " + WiFi.macAddress());
  } else {
    Serial.println("\nWiFi Connection Failed!");
    ESP.restart();
  }
}
void connectToServer() {
  Serial.println("Connecting to server: " + String(host) + ":" + String(port));
  socket.begin(host, port);

  // Connection event
  socket.on("connect", [](const char* payload, size_t length) {
    Serial.println("✅ Connected to Socket.IO server!");
    isConnectedToServer = true;

    // Register device with unique ID
    String registerMsg = "\"" + DEVICE_ID + "\"";
    socket.emit("register_device", registerMsg.c_str());
    Serial.println("🔔 Device registered: " + DEVICE_ID);

  });
}
```

Kết quả

Kết nối ESP client 001

```
Multi-ESP32 Server running on http://localhost:3000
Waiting for ESP32 devices clients...
Client connected: 7MZjRiN4cGt2A8gzAAAA
ESP32 registered: ESP32_001 (7MZjRiN4cGt2A8gzAAAA)
ESP32_001 {
  socketId: '7MZjRiN4cGt2A8gzAAAA',
  lastSeen: 1759420375972,
  data: null
}
```

Kết nối ESP client 002

```
Client connected: LpFDcykPsp_IqP35AAAB
ESP32 registered: ESP32_002 (LpFDcykPsp_IqP35AAAB)
ESP32_001 {
  socketId: '7MZjRiN4cGt2A8gzAAAA',
  lastSeen: 1759420375972,
  data: null
}
ESP32_002 {
  socketId: 'LpFDcykPsp_IqP35AAAB',
  lastSeen: 1759420439932,
  data: null
}
```

- Câu hỏi:
  1) Gửi thông tin trạng thái của client về server theo chu kỳ 5 giây lần.
  2) Viết thêm chức năng gửi lệnh đảo trạng thái led tại GPIO 2 trên các client theo chu kỳ.
  3) Thêm sự kiện xử lý ngắt kết nối cho server, client yêu cầu ngắt kết nối bằng cách ấn phím Boot

Web client

- Giám sát và điều khiển các ESP client thông qua Socket.IO
- Chức năng ban đầu:
  - Giám sát các ESP client đã đăng ký vào mạng
  - Tạo lệnh điều khiển đảo trạng thái LED tại GPIO 2 của client được chọn
  - In dữ liệu giả lập từ client được chọn



Cập nhật thêm mã lệnh

- Server side:

Gửi danh sách thiết bị khi có bất kỳ một client đăng ký vào mạng

```
socket.on('register_device', (deviceId) => {
   // Bổ sung
    webClients.forEach(webSocketId => {
      io.to(webSocketId).emit('device_list', Array.from(devices.keys()));
      console.log(Array.from(devices.keys()));
    });
  });
```

Đăng ký khi có một web client được đăng ký vào mạng

```
socket.on('register_web', () => {
    webClients.add(socket.id);
    console.log('Web client registered:', socket.id);
    // Send current device list to new web client
    socket.emit('device_list', Array.from(devices.keys()));
  });
```

Xử lý sự kiện điều khiển một cơ cấu chấp hành tại Esp client

```
socket.on('control_device', ({ deviceId, command }) => {
    console.log(`Control for ${deviceId}:`, command);

    if (devices.has(deviceId)) {
      const deviceInfo = devices.get(deviceId);
      // Send control to specific device room
      io.to(`device_${deviceId}`).emit('control', command);
      console.log(`Control sent to device ${deviceId}`);
    } else {
      console.log(`Device ${deviceId} not found`);
    }
  });
```

Cập nhật thêm mã lệnh

- Server side:

Minh họa sự kiện nhận dữ liệu từ các client

```
socket.on('status', (msg) => {
    console.log('Client socket:', socket.id);
    console.log(msg);
    webClients.forEach((value)=>{
        socket.to(value).emit('status',msg);
    });

  });
```

Xử lý ngắt kết nối

```
socket.on('disconnect', () => {
    console.log('Client disconnected:', socket.id);
    // Remove from web clients
    webClients.delete(socket.id);

    // Find and remove device
    for (const [deviceId, info] of devices) {
      if (info.socketId === socket.id) {
        devices.delete(deviceId);
        console.log(`Device ${deviceId} removed`);

        // Notify web clients about device removal
        webClients.forEach(webSocketId => {
          io.to(webSocketId).emit('device_list', Array.from(devices.keys()));
          io.to(webSocketId).emit('device_disconnected', deviceId);
        });
        break;
      }
    }
  });
});
```

Cập nhật thêm mã lệnh

- Web client:

```
<body>
  <h1>Multi-ESP32 Dashboard</h1>
  <div class="container">
    <!-- Khu vực chọn thiết bị và điều khiển -->
    <div class="top-section">
      <div class="device-selector">
        <label for="dev_l">Các client đang kết nối:</label>
        <select id="dev_l"></select>
      </div>


      <div class="controls">
        <button id="myBtn">🔘 Toggle LED2</button>
      </div>


      <!-- Hiển thị timestamp -->
      <div class="counter-box">
        <span class="counter-label">Client timestamp:</span>
        <span id="clientTime" class="time-value">0</span>
      </div>
    </div>
```

```
    <!-- Thanh hiển thị số lượng thiết bị -->
    <div class="status-bar">
      <span class="status-label">Số lượng thiết bị:</span>
      <span id="deviceCount">0</span>
    </div>


    <!-- Nút xóa log nằm giữa -->
    <div class="log-controls">
      <button class="button" id="clearLog">
        🧹 Xóa log
      </button>
    </div>


    <!-- Logs -->
    <div class="log-box" id="logBox"></div>
  </div>
```

Web client

```
socket.on('connect', () => {
  logMessage("✅ Kết nối server thành công");
  socket.emit('register_web');
});


// Nhận timestamp từ server
socket.on('status', (msg) => {
  document.getElementById("clientTime").textContent = msg.timestamp;
});


socket.on('disconnect', () => {
  logMessage("❌ Mất kết nối server");
});


// Nhận danh sách thiết bị
socket.on('device_list', (deviceIds) => {
  logMessage("📥 Danh sách thiết bị: " + deviceIds.join(", "));
  const dev_list = document.getElementById("dev_l");
  dev_list.innerHTML = "";
  deviceIds.forEach(deviceId => addOption(deviceId));
  document.getElementById("deviceCount").textContent = deviceIds.length;
});
```

```
// Thêm option có icon
function addOption(deviceId) {
  const dev_list = document.getElementById("dev_l");
  const option = document.createElement("option");
  const icon = getDeviceIcon(deviceId);
  option.value = deviceId;
  option.text = `${icon} ${deviceId}`;
  dev_list.add(option);
}


// Gửi yêu cầu lấy danh sách thiết bị khi load
window.addEventListener('load', () => {
  setTimeout(() => {
    socket.emit('get_devices');
  }, 1000);
});
```

Cập nhật thêm mã lệnh

- ESP client:

```
void loop() {
  socket.loop();
  delay(100);
  // Check WiFi connection
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi disconnected, reconnecting...");
    connectWiFi();
  }


  // Send heartbeat every 10 seconds
  if (millis() - lastStatus > 10000) {
    sendStatus();
    lastStatus = millis();
  }
}
```

```
void sendStatus() {
  if (!isConnectedToServer) return;

  String heartbeat = "{\"type\":\"status\",\"timestamp\":" + String(millis()) + "}";
  socket.emit("status", heartbeat.c_str());
  Serial.println("Status sent");
}
```

ESP client

```
// Control event handler
socket.on("control", [](const char* payload, size_t length) {
  Serial.println("Control received: " + String(payload));
  String command = String(payload);
  // Tạo vùng nhớ cho JSON
  String raw(payload);
  // Loại bỏ ký tự escape "\" nếu có
  raw.replace("\\", "");
  // Tạo vùng nhớ cho JSON object
  StaticJsonDocument<128> doc;
  // Parse chuỗi JSON
  deserializeJson(doc, raw);
  // Lấy giá trị ra
  String value = doc["cmd"];
  Serial.print("Parsed s_data = ");
  Serial.println(value);
  command.toLowerCase();
  if (value == "toggle") {
    ledState = !ledState;
    digitalWrite(ledPin, ledState);
    Serial.println("LED toggled: " + String(ledState ? "ON" : "OFF"));
  } else {
    Serial.println("Unknown command: " + command);
  }
});
```

Bài tập

- 1) Thêm chức năng điều khiển ON/OFF (Button, Image, Check box, Switch)
- 2) Thêm chức năng điều khiển độ sáng của một LED lắp thêm vào esp client (Input text, slider)
- 3) Thay timestamp trong ví dụ bang giá trị điện áp đọc được từ một biến trở, hãy thêm một đồ thị để xem 16 mẫu điện áp lịch sử

Thiết kế ứng dung Web sử dụng Express

- Cài đặt: `npm install express@4`

- Ứng dụng cơ bản:

```
import express from 'express'
const app = express()
const port = 3000
app.get('/', (req, res) => {
    res.send('Hello World from Express!')
})

app.listen(port, () => {
    console.log(`Hello World app listening on port ${port}`)
})
```

- Thực hiện ứng dụng: `node helloWorld.js`

Render a template

- Embedded JavaScript templating - EJS: helloWorldTemplate.js

```
import express from 'express'
const app = express()
const port = 3000
app.set('view engine', 'ejs')
app.get('/', (req, res) => {
    res.render('index', {
        title: 'This is an Express app',
        subtitle: 'using EJS as template engine'
    })
})
app.listen(port, () => {
    console.log(`Application running in http://localhost:${port}`)
})
```

Render a template

- Tạo /views/index.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <title>
                <%= title %>
        </title>
</head>

<body>
        <h1>
                <%= title %>
        </h1>
        <h2>
                <%= subtitle %>
        </h2>
</body>
</html>
```

- Cài ejs: *npm install ejs@3*
- Khởi động ứng dụng: *node helloWorldTemplate.js*

Hoạt động của server khi có một yêu cầu GET

- Báo với Express sử dụng ejs là template để render ứng dụng:

```
app.set('view engine', 'ejs')
```

- Render template đã cài đặt:

```
res.render('index', {
    title: 'This is an Express app',
    subtitle: 'using EJS as template engine'
})
```

- Phương thức này nhận vào hai tham số: 1) tên template đã cài đặt views/index.ejs và 2) là dữ liệu cung cấp dữ liệu cho render

Xử lý yêu cầu

- Mã giả xử lý yêu cầu:

```
import express from ('express')
const app = express()
app.method(route, handler)
```

- Trong đó:

  method: là yêu cầu HTTP;

  route: đường dẫn đến chương trình xử lý

  handler: hàm callback được gọi khi method và route trùng khớp.

```
app.get ('/', () => {})
app.post('/', () => {})
app.put('/', () => {})
app.delete('/', () => {})
```

```
app.all('/', () => {})
```

Route

- Static paths (slide 40)
- Dynamic parameters
- Optional parameters
- Regular expressions
- Query params

```
app.get ('/', () => {})
app.post('/', () => {})
app.put('/', () => {})
app.delete('/', () => {})
```

Dynamic parameters

- Xử lý yêu cầu đến một đường dẫn cụ thể nào đó, dấu : được sử dụng để định nghĩa các tham số động

```
app.get('/users/:id', () => {})
```

':id' là tham số động, có thể là: 1, /dev01, /ESP-001

- Sử dụng phối hợp:

```
app.get('/users/:id/profile', () => {})
```

- Express cung cấp giá trị cho dynamic params trong đối tượng *req.params*:

```
app.get('/users/:id', (req, res) => {
    const { originCity, destinationCity } = req.params
    res.send(`You are flying from ${originCity} to
${destinationCity}`)
})
```

Regular expressions

- Được sử dụng để định nghĩa route, đặc biệt hữu ích trong xử lý các đường dẫn có cụm ký tự giống nhau

```
app.get(/.*fly$/, (req, res) => {
  res.send(`Match with any route that ends with fly`)
})
```

Query params

- Giả sử có yêu cầu: *URL /films?category=scifi&director=George+Lucas*
- Yêu cầu này có hai tham số truy vấn là *category* và *director*
- Các tham số này sẽ được lưu trong đối tượng *req.query*, ví dụ:

```
app.get('/films', (req, res) => {
    const { category, director } = req.query
    res.send(`You are looking for films with category ${category} and
director ${director}`)
})
```

Hàm xử lý yêu cầu Handlers

- Là hàm được gọi khi một yều khớp với route, mẫu hàm xử lý yêu cầu:

```
app.get('/', (req, res, next) => {
    res.send("Hello World")
})
```

  - req: chứa thông tin về yêu cầu: parameters, IP, headers, body, …
  - res: chứa các phương thức xử lý phản hồi cho yêu cầu
  - next: hàm được sử dụng để chuyển yêu cầu đến một hàm xử lý khác

Ví dụ

```
const express = require('express');
const app = express();

// Middleware 1: ghi log request
app.use((req, res, next) => {
    console.log(`Yêu cầu đến: ${req.method} ${req.url}`);
    next(); // chuyển sang middleware kế tiếp
});

// Route chính
app.get('/', (req, res, next) => {
    // Sử dụng req để lấy thông tin
    console.log('Header User-Agent:', req.headers['user-agent']);
    // Gửi phản hồi về client
    res.send("Hello World");

    // Nếu muốn tiếp tục chạy middleware
    // khác sau khi phản hồi (ít dùng)
    // next();
});

// Middleware 2: xử lý lỗi hoặc ghi log sau phản hồi
app.use((req, res) => {
    console.log('Hoàn tất xử lý yêu cầu');
});

app.listen(3000, () => {
    console.log('Server chạy tại http://localhost:3000');
});
```

Phản hồi yêu cầu

- Các chủ đề: adding headers, status codes, redirects, sending data
- Quản lý header:
  - Thiết lập header bằng hàm set

```
app.get('/', (req, res, next) => {
    res.set('Content-Type', 'text/html')
    res.send("<h1>Hello World</h1>")
})
```

  - Thiết lập nhiều header cùng lúc

```
app.get('/', (req, res, next) => {
  res.set({
    'Cache-Control': 'no-store', // Không cho cache dữ liệu
    'Server': 'Express'          // Hiển thị thông tin server
  });

  res.send('<h1>Hello World</h1>');
});
```

Mã trạng thái

- Con số thể hiện trạng thái phản hồi của máy chủ

```
app.get('/', (req, res, next) => {
    res.status(200)
    res.send("<h1>Hello World</h1>")
})
```

```
app.get('/', (req, res, next) => {
    res.status(200).set('Content-Type', 'text/html').send("<h1>Hello
World</h1>")
})
```

Redirect

- Chuyển yêu cầu đến một URL khác

```
app.get('/', (req, res, next) => {
    res.redirect('https://ulisesgascon.com/')
})
```

```
app.get('/', (req, res, next) => {
    res.redirect(301, 'https://ulisesgascon.com/')
})
```

```
app.get('/', (req, res, next) => {
    res.redirect('/about')
})
```

```
app.get('/about/me', (req, res, next) => {
    res.redirect('..')
})
```

Gửi dữ liệu

```
app.get('/', (req, res, next) => {
    res.send("Hello World")
})
```

```
app.get('/', (req, res, next) => {
    res.json({message: 'Hello World'})
})
```

```
app.get('/', (req, res, next) => {
    res.set('Content-Type', 'text/html')
    res.send(Buffer.from('<p>Hello World</p>'))
})
```

```
app.get('/', (req, res, next) => {
    res.set('Content-Type', 'application/json')
    res.send(JSON.stringify({message: 'Hello World'}))
})
```

Cám ơn